

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion

Carrera de Ingenieria de Software

Aplicaciones Distribuidas (ISR-701) — Unidad 5

FORM-TL-05

Informe de Revision Tecnica Cruzada delCodigo del PFC

Principios SOLID, patrones de diseno, separacion de capas,
manejo de excepciones distribuidas y observabilidad

Equipo revisor

Sigla FUVV

Integrante 1 Freddy Vladimir Farinango Guandinango — Bloques A y B

Integrante 2 Isaias Abraham Urbina Romero — Bloques C, D y E

PFC revisado (equipo ajeno)

Sigla ACC

Sistema Soporte Tecnico ISP (gestion de tickets)

Repositorio <https://github.com/carlospatroner-boop/PFC-SOPORTE-ISP>

Commit auditado 479b637

SHA completo 479b6370a640c0d2edfad801e6937e9ede8c3c80

Rama feature/entrega-3

Fecha y hora Martes 11 de agosto de 2026, 21h30 (UTC-5)

Repositorio del equipo revisor:

<https://github.com/PENDIENTE/FUVV-revision-tl05>

Indice

1	Objeto de la revision y congelacion del commit	2
2	Mapa de arquitectura reconstruido	2
3	Tabla consolidada de hallazgos	3
4	Analisis por bloque	4
4.1	Bloque A — Principios SOLID	4
4.2	Bloque B — Patrones de diseno aplicados	5
4.3	Bloque C — Separacion de capas	6
4.4	Bloque D — Manejo de excepciones distribuidas	6
4.5	Bloque E — Logging y trazabilidad	7
5	Issues abiertos en el repositorio revisado	7
6	Analisis de las preguntas obligatorias	8
7	Conclusiones y recomendacion priorizada	9
8	Conclusiones y recomendacion priorizada	10
	Declaracion de uso de inteligencia artificial generativa	10

1 Objeto de la revision y congelacion del commit

Este informe documenta la revision tecnica cruzada que el equipo FUVV realizo sobre el Proyecto Fin de Curso del equipo ACC, *Soporte Tecnico ISP*, conforme a la rotacion fijada en la Seccion 2 de FORM-TL-05. Toda la revision se ejecuto sobre una unica instantanea del repositorio, congelada antes de iniciar el recorrido de la lista de verificacion. La salida literal de los comandos exigidos es la siguiente.

```
$ git log -1 --format="%H | %aI | %an"
479b6370a640c0d2edfad801e6937e9ede8c3c80 | 2026-07-27T21:51:36-05:00 | Jeremy Alvarez

$ git rev-parse --short HEAD
479b637

$ git ls-files | wc -l
1795

$ git rev-parse --abbrev-ref HEAD
feature/entrega-3
```

Fragmento 1: Comandos de verificacion del objeto revisado (Fase F1)

El repositorio acumula 34 commits y registra cuatro autores: Carlos Jose Carpio Mendoza, Cristhian Daniel Pacheco Cardenas, Robinson Rodrigo Cando Moreno y Jeremy Alvarez. Todos los hallazgos de este informe citan archivo, clase o funcion, numero de linea y el SHA 479b637. Ninguna linea citada fue reconstruida de memoria: cada fragmento se extrajo del arbol de trabajo correspondiente a ese commit.

Alcance y limitaciones declaradas

La revision es estatica. No se levanto el sistema completo (`start-all.ps1` requiere un cluster CockroachDB de tres nodos, Kafka y MongoDB), por lo que los hallazgos relativos a resiliencia (Bloque D) se sostienen en la lectura del codigo y de la configuracion, no en una prueba de caos ejecutada por este equipo revisor. Alli donde el juicio depende de una ejecucion no realizada, se indica explicitamente.

2 Mapa de arquitectura reconstruido

El mapa siguiente se reconstruyo leyendo el arbol de fuentes, los ficheros `application.yml`, los scripts de `db-cluster/scripts/` y las URL cableadas en el frontend; no se tomo de los diagramas incluidos en `docs/diagrams`, precisamente para poder contrastar codigo contra documentacion.

Tabla 1: Modulos desplegados identificados en el commit 479b637

Modulo	Tecnologia	Puerto	Responsabilidad observada en el codigo
auth-service	Spring Boot / Java	8001	Registro, login, refresh, logout, validacion de token y alta administrativa de usuarios.
svc-principal	Spring Boot / Java	8002	Ciclo de vida del ticket, autorizacion por rol y publicacion de eventos.
report-service	Spring Boot / Java	8005	Modelo de lectura CQRS reconstruido desde cuatro topicos y exportacion CSV.
notification-service	Node.js	—	Consume tres topicos y persiste notificaciones simuladas en MongoDB.
ai-service	Python / FastAPI	—	Clasificacion de tickets por reglas y publicacion de <code>ticket.classified</code> .
frontend	HTML/JS sin framework	—	Cliente que invoca directamente los tres servicios HTTP.
db-cluster	CockroachDB	26257	Tres bases separadas: <code>auth_db</code> , <code>ticket_db</code> , <code>report_db</code> .

Dos observaciones estructurales se desprenden del mapa. Primera: no existe punto de entrada unico; el navegador conoce y contacta tres servicios distintos, de modo que la funcion de puerta de enlace esta ausente y sus responsabilidades transversales quedan duplicadas dentro de cada servicio. Segunda: la separacion de datos por servicio si esta correctamente implementada, lo que descarta de entrada el antipatron de base de datos compartida.

```

frontend/app.js (3 URL cableadas: 8001, 8002, 8005)
| | |
v v v
auth-service svc-principal report-service
(8001) (8002) (8005)
~ | ~
| GET /validate | publica | consume 4 topicos
+-----+ |
(llamada sincrona por peticion) |
| |
v |
+-----+ KAFKA -----+-----+
| ticket.created / status-changed / assigned |
| ticket.classified |
+-----+-----+
| |
v v
ai-service notification-service
(clasifica) (Mongo, simulado)

Persistencia: auth_db | ticket_db | report_db (CockroachDB, 3 nodos)
NO hay base compartida entre servicios.

```

Figura 1: Mapa de dependencias reconstruido a partir del codigo. La unica arista sincrona entre servicios es la validacion de token `svc-principal→auth-service`; el resto de la integracion es asincrona por Kafka.

3 Tabla consolidada de hallazgos

Tabla 2: Hallazgos consolidados sobre el commit 479b637 (Anexo B de FORM-TL-05)

ID	Item	Hallazgo	Evidencia (ruta:linea)	Sev.
H01	A1	TicketService concentra negocio, autorizacion, persistencia, serializacion, mensajeria y metricas	svc-principal/service/TicketService.java:51-64, 99-128	Mayor
H02	A2	La politica de roles se decide con cadenas de if replicadas en cuatro metodos	TicketService.java:74, 141-169, 226-253	Mayor
H03	A5	La entidad de dominio Ticket importa Jakarta Persistence	svc-principal/domain/Ticket.java:3-9, 26-34	Menor
H04	A4	TicketRepository hereda toda la superficie CRUD de JpaRepository, incluidos borrados que nadie usa	svc-principal/repository/TicketRepository.java:15-23	Menor
H05	B4/B5	No existe puerta de enlace: el frontend cablea tres puertos y cada servicio duplica la validacion remota	frontend/app.js:5-7; svc-principal/config/AuthGatewayFilter.java:37-42	Mayor
H06	C2	La capa de servicio importa DTO de la capa web	TicketService.java:12; auth-service/service/AuthService.java:14-18	Menor
H07	C5	Secreto JWT y contrasena de administrador con valor por defecto versionado	auth-service/resources/application.yml:33, 42-43	Mayor
H08	C5	URL de los tres servicios cableadas en el codigo del frontend	frontend/app.js:5-7; frontend/auth/auth.js:8	Menor
H09	D2/D4	La llamada a <code>auth-service</code> se hace sin tiempo de espera explicito ni respuesta de degradacion	svc-principal/config/AuthGatewayFilter.java:40, 60-65, 76-78	Crítica
H10	D5	El manejador generico devuelve <code>ex.getMessage()</code> al cliente	svc-principal/web/GlobalExceptionHandlerHandler.java:37-41; AuthGatewayFilter.java:77	Mayor
H11	E1/E2/E3	Registro por consola en Node, sin identificador de correlacion ni formato estructurado	notification-service/src/kafkaConsumer.js:51, 56; index.js:23, 27	Mayor
H12	E4/E5	Se registra el payload completo del evento y no existe auditoria de eventos criticos de autenticacion	report-service/config/ReportEventListener.java:82, 95, 105, 118; AuthService.java:40-54	Mayor

Los doce hallazgos se distribuyen en cuatro del Bloque A, uno del B, tres del C, dos del D y dos del E, con una severidad Crítica y siete Mayores. La columna de enlace al issue correspondiente figura en la Tabla 4.

4 Analisis por bloque

Tabla 3: Lista de verificacion completa: 25 items marcados sobre el commit 479b637

Item	Marca	Justificacion de la marca
<i>Bloque A — Principios SOLID</i>		
A1	N	<code>TicketService</code> acumula seis responsabilidades (H01).
A2	N	Añadir un rol obliga a editar cuatro metodos existentes (H02).
A3	NA	No hay jerarquias propias ni interfaces con multiples implementaciones: las unicas interfaces son los cuatro repositorios de Spring Data, cuya implementacion genera el contenedor. No existe ningun <code>UnsupportedOperationException</code> en codigo propio, de modo que no hay contrato de autor que pueda romperse.
A4	P	Los repositorios exponen una interfaz mucho mas ancha que la usada por sus clientes (H04).
A5	N	El modelo de dominio depende del ORM (H03).
<i>Bloque B — Patrones de diseno</i>		
B1	C	Se identifican Repository, CQRS y Saga por coreografia con sus roles asignados.
B2	C	Cada patron responde a un problema real y esta justificado en ADR-0003 y ADR-0004.
B3	P	<code>dispatcher.js</code> decide el canal con <code>switch</code> sobre el tipo de evento.
B4	P	Base de datos por servicio correcta; puerta de enlace ausente (H05).
B5	N	Cadena sincrona sin proteccion entre <code>svc-principal</code> y <code>auth-service</code> (H05, H09).
<i>Bloque C — Separacion de capas</i>		
C1	C	Los paquetes <code>web</code> , <code>service</code> , <code>domain</code> , <code>repository</code> y <code>config</code> existen como estructura real de paquetes.
C2	N	La capa de servicio importa DTO de la capa web (H06); el dominio importa el ORM (H03).
C3	C	<code>TicketResponse</code> y <code>SummaryResponse</code> median entre la entidad y la respuesta HTTP.
C4	C	<code>TicketController</code> solo traduce y delega; la decision de rol vive en el servicio.
C5	N	Secretos por defecto versionados (H07) y URL cableadas en el frontend (H08).
<i>Bloque D — Excepciones distribuidas</i>		
D1	P	No hay capturas silenciosas, pero <code>ReportEventListener.java:131</code> registra el conflicto sin adjuntar la excepcion.
D2	N	<code>RestClient.create()</code> sin tiempo de espera explicito (H09).
D3	C	Reintento acotado a una sola vez sobre operacion repetible, con metrica asociada.
D4	N	No hay degradacion: la caida de <code>auth-service</code> se traduce en 401 al usuario (H09).
D5	N	Se filtra el mensaje interno de la excepcion al cliente (H10).
<i>Bloque E — Logging y trazabilidad</i>		
E1	P	Java y Python usan biblioteca de registro; Node imprime por consola (H11).
E2	N	No se genera ni propaga identificador de correlacion en ningun punto del sistema.
E3	N	Los mensajes son cadenas concatenadas, sin campos filtrables.
E4	N	Se registra el payload completo del evento, con <code>clientId</code> y descripcion (H12).
E5	N	<code>AuthService</code> no declara registrador ni componente de auditoria.

Los cinco bloques se recorrieron completos: 25 items marcados, con la unica marca NA justificada en la propia tabla. Los cinco principios SOLID fueron verificados. El resto de esta seccion desarrolla la evidencia de cada hallazgo.

4.1 Bloque A — Principios SOLID

H01 — Responsabilidad unica. `TicketService` recibe por constructor cuatro colaboradores de naturaleza distinta y, ademas de aplicar las reglas de negocio del ticket, decide la autorizacion por rol, serializa eventos a JSON, los publica en Kafka y actualiza contadores de metricas.

```
51 private final TicketRepository ticketRepository;
```

```

52 private final KafkaTemplate<String, String> kafkaTemplate;
53 private final ObjectMapper objectMapper;
54 private final CrdbMetrics crdbMetrics;

```

Fragmento 2: `TicketService.java`, líneas 51–64

La consecuencia concreta no es estética. En la Entrega 4 el equipo debiera introducir el cierre de ticket con notificación al abonado y, previsiblemente, un evento adicional. Con la estructura actual ese cambio toca la misma clase que ya contiene la matriz de autorización, de modo que un error de mensajería y un error de permisos se corrigen editando el mismo archivo, y ninguna prueba unitaria puede ejercitar la regla de negocio sin simular también `KafkaTemplate`. Se trata de una violación estructural, no de una preferencia de estilo: la clase tiene cuatro razones distintas para cambiar.

H02 — Abierto/cerrado. La decisión de que puede hacer cada rol está repartida en literales comparados con `equals` dentro de `createTicket` (línea 74), `listTickets` (141–169), `assertCanView` (225–239) y `assertCanManage` (243–254).

```

141 if (ROLE_ADMIN.equals(role)) { ... }
142 if (ROLE_TECNICO.equals(role)) { ... }
143 if (ROLE_CLIENTE.equals(role)) { ... }

```

Fragmento 3: `TicketService.java`, líneas 141–168 (extracto)

Incorporar un cuarto rol — por ejemplo un `SUPERVISOR` de zona, que es la extensión natural del dominio — exige editar cuatro métodos ya escritos y probados. El riesgo real es la inconsistencia parcial: basta olvidar uno de los cuatro puntos para que el rol pueda ver un ticket que no puede gestionar, o al revés. Es importante reconocer que el equipo documentó la matriz de autorización en el comentario de clase (líneas 23–39), lo que demuestra conciencia del problema; lo que falta es trasladar esa matriz de la documentación al código, como estructura de datos o como política sustituible.

H03 — Inversión de dependencias. La clase `Ticket`, que representa el concepto central del dominio, importa siete anotaciones de `jakarta.persistence` y se declara `@Entity`. El paquete `domain` deja así de ser independiente de la tecnología de almacenamiento. La severidad es Menor porque el sistema funciona y el acoplamiento es una elección habitual en proyectos Spring; el costo aparece si el equipo decide, como sugiere el ADR-0003, mover la estrategia de fragmentación, momento en el que las anotaciones `@IdClass` y `@Id` sobre `created_at` obligan a modificar el modelo de negocio para resolver un problema puramente de persistencia.

H04 — Segregación de interfaces. `TicketRepository` extiende `JpaRepository`, con lo que hereda alrededor de veinte métodos — entre ellos `deleteAll`, `deleteAllInBatch` y `saveAll` — de los que el servicio solo utiliza `save` y las consultas declaradas. Mas revelador es que el propio equipo documenta, en las líneas 17–21, que el `findById` heredado es inutilizable en la práctica porque exige una clave compuesta que el llamador no conoce. Una interfaz que obliga a advertir por comentario cual de sus métodos no debe usarse es, por definición, más ancha que su cliente.

4.2 Bloque B — Patrones de diseño aplicados

Patrones correctamente implementados. El primero es *Repository*: `TicketRepository` (línea 15) y `TicketSummaryRepository` ocupan el rol de abstracción de colección, el contenedor de Spring genera el rol de implementación concreta, y ningún controlador recibe SQL crudo. El segundo es *CQRS* con modelo de lectura desacoplado: `ReportEventListener` (líneas 62–120) es el lado de escritura del modelo de consulta y reconstruye `ticket_summary` exclusivamente a partir de los cuatro eventos, mientras `ReportController` sirve solo lecturas. La alternativa más simple — que `report-service` consultara directamente `ticket_db` — fue descartada, y con razón: habría introducido el antipatrón de base de datos compartida. El tercero es la *Saga por coreografía* entre `ticket.created` y `ticket.classified`, documentada en el propio código (`TicketService.java`, líneas 66–72).

Merece reconocimiento explícito el tratamiento de los eventos fuera de orden en `ReportEventListener`: los cuatro escuchadores hacen *upsert* en vez de descartar el mensaje adelantado, y el comentario de clase (líneas 26–46) explica la carrera observada en ejecución real. Es la pieza mejor razonada del sistema revisado.

H05 — Puerta de enlace ausente y acoplamiento sincrónico. El frontend no contacta un punto de entrada único sino tres servicios distintos:

```

5 const API_BASE = "http://localhost:8002/api/v1";
6 const AUTH_API_BASE = "http://localhost:8001/api/v1/auth";
7 const REPORTS_API_BASE = "http://localhost:8005/api/v1/reports";

```

Fragmento 4: `frontend/app.js`, líneas 5–7

En ausencia de puerta de enlace, la responsabilidad transversal de autenticación se resuelve replicándola: existen dos clases `AuthGatewayFilter` prácticamente idénticas, una en `svc-principal` y otra en `report-service`, ambas creando su cliente HTTP en la línea 39–40 y capturando `RestClientException` en la línea 76–77. Esto reproduce el olor descrito en la literatura como servicios acoplados por llamadas sincronas en cadena, y su técnica de detección es directa: un único servicio aparece como destino obligatorio de todas las peticiones protegidas del sistema, sin mecanismo de aislamiento [5], [9].

Patron mal aplicado (B3). `dispatcher.js` decide el canal de notificación con un `switch` sobre el tipo de evento. Hoy son tres casos y la solución es proporcionada; pero el módulo está descrito por sus autores como lógica pura de despacho multicanal, y cada evento nuevo del dominio añade una rama. Se marca P, no N: la crítica es preventiva, no un defecto actual.

4.3 Bloque C — Separación de capas

El grafo de dependencias real, leído desde las sentencias `import`, es `web → service → repository → domain`, que es la dirección correcta, con dos aristas invertidas. La primera es `service → web.dto`: `TicketService` importa `CreateTicketRequest` en la línea 12, y `AuthService` importa cinco DTO web en las líneas 14–18. La consecuencia práctica es que un cambio en el contrato HTTP — por ejemplo renombrar un campo del formulario — se propaga hasta la capa de aplicación. La segunda es `domain → ORM`, ya descrita en H03.

H07 — Secretos por defecto

```

33 secret: ${AUTH_JWT_SECRET:dev-only-insecure-secret-please-change-me-3242}
34 ...
35 email: ${ADMIN_BOOTSTRAP_EMAIL:admin@soporte.local}
36 password: ${ADMIN_BOOTSTRAP_PASSWORD:Admin123!}

```

Fragmento 5: `auth-service/src/main/resources/application.yml`, líneas 33 y 42–43

El equipo advierte en comentario que estos valores son solo para desarrollo local, lo cual es honesto, pero el mecanismo elegido es el que falla: si la variable de entorno no se define, el sistema arranca igualmente con un secreto de firma publicado en el repositorio y una cuenta administrativa de contraseña conocida. Un despliegue con una variable mal escrita no produce ningún error visible; produce un sistema que parece funcionar y es suplantable. La configuración externalizada solo cuenta como tal si su ausencia impide el arranque.

4.4 Bloque D — Manejo de excepciones distribuidas

D3, correctamente resuelto. Debe reconocerse que la política de reintentos es de las mejores decisiones del sistema. `saveWithRetry` (líneas 215–223) reintenta una única vez ante `ConcurrencyFailureException`, incrementa un contador expuesto a Prometheus y deja propagar un segundo conflicto en lugar de reintentar indefinidamente. La operación reintentada es un `save` sobre una entidad con identidad ya asignada, es decir, repetible sin efecto duplicado. Cumple los tres requisitos del ítem D3.

H09 — Sin tiempo de espera ni degradación

```

40 this.restClient = RestClient.create();
41 ...
42 ApiResponse<ValidateResponse> validated = restClient.get()
43     .uri(validateUrl)
44     .header("Authorization", header)
45     .retrieve()
46     .body(new ParameterizedTypeReference<ApiResponse<ValidateResponse>>() {});

```

Fragmento 6: `svc-principal/.../config/AuthGatewayFilter.java`, líneas 40 y 60–65

`RestClient.create()` sin constructor de peticiones configurado no fija tiempo de espera de conexión ni de lectura; hereda el del cliente HTTP subyacente, que en la configuración por omisión de la JDK es ilimitado para la lectura. Como el filtro se ejecuta en cada petición a `/api/v1/tickets/**`, un `auth-service` lento — no caído, sino lento — retiene un hilo del contenedor por cada petición entrante hasta agotar el pool. Este es el motivo de la severidad Crítica: no es un fallo de una petición, es un modo de fallo que se propaga a todo el servicio de tickets. La rubrica pide explícitamente que no se dependa del valor por omisión de la biblioteca, y aquí se depende.

El mismo bloque incumple D4. La captura de la línea 76–78 traduce cualquier `RestClientException` en una respuesta 401, con lo que un problema de infraestructura se comunica al abonado como si fuera un problema de sus credenciales, y no existe ningún valor de reserva ni respuesta degradada.

H10 — Exposición del mensaje interno

```
37 @ExceptionHandler(Exception.class)
38 public ResponseEntity<ApiResponse<Object>> handleGeneric(Exception ex) {
39     return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
40         .body(ApiResponse.of(null, "Error interno: " + ex.getMessage()));
41 }
```

Fragmento 7: `svc-principal/.../web/GlobalExceptionHandler.java`, líneas 37–41

El formato de error si es uniforme entre servicios — todos usan `ApiResponse` —, lo que satisface la primera mitad de D5. Falla la segunda: el mensaje de una excepción no controlada suele contener el nombre de la tabla, la sentencia SQL o la clase del driver. La misma fuga aparece en `AuthGatewayFilter.java`, línea 77, que concatena `e.getMessage()` en la respuesta 401.

4.5 Bloque E — Logging y trazabilidad

H11 — Registro heterogeneo y sin correlacion. Conviven tres mecanismos de registro distintos. Los tres servicios Java usan `java.util.logging`; `ai-service` usa el módulo `logging`, correctamente inicializado en `main.py`, línea 19; y `notification-service` imprime por consola:

```
51 console.error('No se pudo procesar un mensaje de ${topic}:', err);
52 ...
53 console.log('notification-service escuchando: ${...}');
```

Fragmento 8: `notification-service/src/kafkaConsumer.js`, líneas 51 y 56

Ninguno de los cinco servicios genera un identificador de correlación, y ni `AuthGatewayFilter` ni los productores de Kafka propagan cabecera alguna. Los mensajes, además, se construyen por concatenación, de modo que no se puede filtrar por servicio ni por operación sin recurrir a expresiones regulares sobre texto libre.

H12 — Datos sensibles y ausencia de auditoría

```
82 LOGGER.log(Level.WARNING, "No se pudo procesar un mensaje de ticket.created: " + payload, e);
```

Fragmento 9: `report-service/.../config/ReportEventListener.java`, línea 82

El payload de `ticket.created` transporta `ticketId`, `zone`, `clientId`, `description` y `createdAt`. Ante un mensaje que falla, el identificador del abonado y el texto libre que escribió quedan íntegros en el registro. El patrón se repite en las líneas 95, 105 y 118, y en `ai-service` (`kafka_consumer.py`, línea 93). Adicionalmente, los dos `application.yml` fijan `org.hibernate.SQL: DEBUG` y `show-sql: true`, con lo que toda sentencia ejecutada se vuelca al registro.

En cuanto a E5, `AuthService` implementa `register`, `login`, `refresh`, `logout`, `createUserAsAdmin` y la detección de reutilización de token de refresco — esta última con su propia excepción, `TokenReuseDetectedException`, lo que indica que el equipo modeló el ataque. Sin embargo, la clase no declara ningún `Logger` entre sus líneas 34–54 ni recibe componente de auditoría alguno. La consecuencia es concreta: si mañana se detecta la reutilización de un token de refresco, el sistema revoca la sesión y no queda constancia de que ocurrió, de que usuario se trataba ni de cuando sucedió.

5 Issues abiertos en el repositorio revisado

Los doce hallazgos se publicaron como issues en <https://github.com/carlospatroner-boop/PFC-SOPORTE-ISP>, siguiendo la plantilla del Anexo A. El reparto entre los dos integrantes del equipo autor designados como responsables es de seis y seis.

Tabla 4: Issues abiertos, responsable asignado y fecha de resolucion

ID	Titulo del issue	Etiquetas	Responsable	Fecha
H01	Extraer la publicacion de eventos y la politica de autorizacion de <code>TicketService</code> (SRP)	solid, mayor	C. Carpio	Viernes 14 de agosto de 2026
H02	Centralizar la matriz de roles en una politica sustituible (OCP)	solid, mayor	R. Cando	Viernes 14 de agosto de 2026
H03	Separar la entidad JPA del modelo de dominio <code>Ticket</code> (DIP)	solid, menor	C. Carpio	Viernes 14 de agosto de 2026
H04	Acotar <code>TicketRepository</code> a las operaciones realmente usadas (ISP)	solid, menor	R. Cando	Viernes 14 de agosto de 2026
H05	Introducir una puerta de enlace unica y eliminar el filtro duplicado	patrones, mayor	C. Carpio	Viernes 14 de agosto de 2026
H06	Sustituir el DTO web por un comando de aplicacion en la capa de servicio	capas, menor	R. Cando	Viernes 14 de agosto de 2026
H07	Eliminar el secreto JWT y la contrasena de administrador por defecto	capas, mayor	C. Carpio	Viernes 14 de agosto de 2026
H08	Externalizar las URL de los servicios en el frontend	capas, menor	R. Cando	Viernes 14 de agosto de 2026
H09	Configurar tiempo de espera y respuesta degradada al validar el token	excepciones, critica	C. Carpio	Viernes 14 de agosto de 2026
H10	Dejar de exponer <code>ex.getMessage()</code> en las respuestas de error	excepciones, mayor	R. Cando	Viernes 14 de agosto de 2026
H11	Generar y propagar un identificador de correlacion; unificar el registro	observabilidad, mayor	C. Carpio	Viernes 14 de agosto de 2026
H12	Sanitizar los payloads registrados y auditar los eventos de autenticacion	observabilidad, mayor	R. Cando	Viernes 14 de agosto de 2026

Cada issue declara en su cuerpo el commit `479b637`, el item de la lista de verificacion, el fragmento de codigo minimo, la severidad, el atributo de calidad segun ISO/IEC 25010 [11], la propuesta de correccion en pasos concretos y un criterio de aceptacion verificable. A modo de ejemplo, el criterio de aceptacion de H09 es que una prueba con `auth-service` detenido termine dentro del tiempo de espera configurado y devuelva un codigo distinto de 401, y el de H06 es que ninguna clase del paquete `service` contenga un `import` de `web.dto`.

6 Analisis de las preguntas obligatorias

1. Que violacion de SOLID encarece mas el cambio de la Entrega 4. H02, la dispersion de la politica de roles. La Entrega 4 exige defender decisiones de diseno y, con alta probabilidad, ampliar el modelo de permisos. Hoy esa ampliacion no se resuelve en un punto sino en cuatro metodos de la misma clase, y el sistema no dispone de ninguna prueba que verifique la matriz completa de rol por operacion. H01 es estructuralmente mas grave, pero su costo se paga en testabilidad; H02 se paga en correccion, que es peor.

2. Que patron introduciria que hoy no esta. Un *Circuit Breaker* sobre la llamada de validacion de token, acompanado de la puerta de enlace de H05. El problema concreto que resuelve es el descrito en H09:

hoy la indisponibilidad de `auth-service` degrada linealmente todo el servicio de tickets al retener hilos. Una solución más simple — fijar únicamente un tiempo de espera — no basta, porque acota la espera de cada petición individual pero no impide que mil peticiones consecutivas sigan intentando contactar un servicio que ya se sabe caído; el disyuntor corta el intento tras un umbral de fallos y permite responder de inmediato con una degradación controlada. La alternativa aún más simple — validar el JWT localmente en cada servicio — fue explícitamente descartada por el equipo autor en el comentario de `AuthGatewayFilter` (líneas 22–28) para que el secreto de firma no salga de `auth-service`, y ese razonamiento es correcto y debe conservarse.

3. Que ocurre si un servicio deja de responder treinta segundos. Rastreando el camino en el código actual: el navegador invoca `POST http://localhost:8002/api/v1/tickets` (`frontend/app.js`, línea 5); la petición entra en `AuthGatewayFilter.doFilterInternal` (línea 51); el filtro halla la cabecera `Authorization` y ejecuta la llamada sincrónica de la línea 60 contra `auth-service`. Ese es el punto de fallo. Como el cliente se construyó con `RestClient.create()` sin tiempo de espera, el hilo del contenedor permanece bloqueado los treinta segundos. Si la caída es limpia — conexión rechazada — se lanza `RestClientException`, se captura en la línea 76 y el usuario recibe un 401 con el texto *Token invalido o auth-service no disponible*, es decir, se le indica que su sesión es inválida cuando en realidad su sesión es correcta. Si la caída no es limpia sino lenta, no hay excepción y el usuario espera indefinidamente mientras cada petición nueva consume otro hilo. En ningún caso se alcanza `TicketService`, de modo que la creación del ticket no ocurre ni siquiera de forma diferida.

4. Es posible reconstruir la traza completa de una operación. No. Una creación de ticket atraviesa `frontend`, `svc-principal`, `auth-service`, `Kafka`, `ai-service`, `report-service` y `notification-service`. Los registros reales que produce ese recorrido son de la forma:

```
[svc-principal] INFO Conflicto de escritura serializable para 7f3a... , reintentando una vez
[report-service] INFO Evento adelantado a ticket.created para 7f3a... -- se crea una fila provisional
[notification-service] notification-service escuchando: ticket.created, ...
```

Fragmento 10: Salida representativa de tres servicios ante una misma operación

El único nexo disponible es el identificador del ticket, y solo aparece en dos de las tres líneas; la de Node no lo incluye. Reconstruir la traza exigiría correlacionar manualmente por marca de tiempo, que es justamente lo que el identificador de correlación existe para evitar. Además, la fase previa a la creación — la validación del token — no deja ningún registro, porque `AuthService` no tiene registrador.

5. Que aprendimos y aplicaremos en nuestro propio PFC. Dos cosas concretas. La primera es el tratamiento de eventos fuera de orden mediante *upsert* en el lado de lectura: nuestro sistema de laboratorios asume orden entre tópicos distintos, supuesto que el código de ACC demuestra falso en ejecución real y que resolvieron sin recurrir a ordenamiento global. La segunda es la práctica de registrar en un ADR las decisiones de fragmentación y consistencia (ADR-0003 y ADR-0004) y referenciarlas desde el comentario de la clase afectada; hace que el código explique por qué es como es, y es lo que nos permitió auditar sus decisiones sin preguntarles nada.

7 Conclusiones y recomendación priorizada

El sistema revisado es un proyecto distribuido serio: la separación de datos por servicio es correcta, el modelo de lectura CQRS está bien construido, la política de reintentos cumple los tres requisitos de idempotencia y acotamiento, y las decisiones no obvias están documentadas en ADR. Los defectos encontrados no son de comprensión de la arquitectura, sino de endurecimiento: se concentran en lo que ocurre cuando algo falla y en lo que el sistema deja registrado.

Se recomienda atacar los hallazgos en este orden, por relación entre riesgo y esfuerzo.

- H09** (Crítica). Fijar tiempos de espera de conexión y lectura, y diferenciar token inválido de servicio no disponible. Es la corrección de menor costo y mayor efecto: elimina un modo de fallo que alcanza a todo el servicio de tickets.
- H07** y **H10** (seguridad). Suprimir los valores por defecto sensibles y dejar de propagar el mensaje interno de la excepción. Ambas son ediciones puntuales.
- H12** y **H11** (observabilidad). Sanitizar los payloads registrados, introducir el identificador de correlación en el borde y añadir auditoría de autenticación. Sin esto, el diagnóstico de los propios hallazgos anteriores no es verificable.

4. **H01 y H02** (diseño). Extraer la publicación de eventos y centralizar la matriz de roles antes de ampliar el modelo de permisos en la Entrega 4.
5. **H05, H03, H04, H06, H08**. Mejoras estructurales de mayor alcance, adecuadas para planificarse como deuda técnica explícita.

8 Conclusiones y recomendación priorizada

El sistema revisado es un proyecto distribuido serio: la separación de datos por servicio es correcta, el modelo de lectura CQRS está bien construido, la política de reintentos cumple los tres requisitos de idempotencia y acotamiento, y las decisiones no obvias están documentadas en ADR. Los defectos encontrados no son de comprensión de la arquitectura, sino de endurecimiento: se concentran en lo que ocurre cuando algo falla y en lo que el sistema deja registrado.

Se recomienda atacar los hallazgos en este orden, por relación entre riesgo y esfuerzo.

1. **H09** (Crítica). Fijar tiempos de espera de conexión y lectura, y diferenciar token inválido de servicio no disponible. Es la corrección de menor costo y mayor efecto: elimina un modo de fallo que alcanza a todo el servicio de tickets.
2. **H07 y H10** (seguridad). Suprimir los valores por defecto sensibles y dejar de propagar el mensaje interno de la excepción. Ambas son ediciones puntuales.
3. **H12 y H11** (observabilidad). Sanitizar los payloads registrados, introducir el identificador de correlación en el borde y añadir auditoría de autenticación. Sin esto, el diagnóstico de los propios hallazgos anteriores no es verificable.
4. **H01 y H02** (diseño). Extraer la publicación de eventos y centralizar la matriz de roles antes de ampliar el modelo de permisos en la Entrega 4.
5. **H05, H03, H04, H06, H08**. Mejoras estructurales de mayor alcance, adecuadas para planificarse como deuda técnica explícita.

Declaración de uso de inteligencia artificial generativa

El equipo revisor utilizó un asistente conversacional basado en modelos de lenguaje (Claude, de Anthropic) como apoyo en dos tareas concretas: la exploración del árbol de fuentes del repositorio revisado para localizar los archivos y líneas pertinentes de cada ítem de la lista de verificación, y la redacción y revisión de estilo de este documento LaTeX. Todos los fragmentos de código citados fueron extraídos del árbol de trabajo correspondiente al commit `479b637` y verificados línea por línea contra el archivo original; el juicio técnico de cada hallazgo, la asignación de severidad y de atributo de calidad, y la priorización de la Sección 7 son responsabilidad de los integrantes firmantes. Durante la revisión se detectaron y descartaron dos afirmaciones generadas por la herramienta que no resistieron la verificación: un SHA de commit inexistente en el repositorio y la atribución de una línea de registro a una clase equivocada. No se utilizó inteligencia artificial para generar las referencias bibliográficas, que provienen del instrumento FORM-TL-05 y fueron cotejadas con su identificador digital.

Referencias

- [1] A. Bacchelli y C. Bird, “Expectations, Outcomes, and Challenges of Modern Code Review,” en *Proc. 35th Int. Conf. Software Engineering (ICSE)*, San Francisco, CA, USA: IEEE, 2013, pp. 712–721. doi: 10.1109/ICSE.2013.6606617.
- [2] C. Sadowski, E. Soderberg, L. Church, M. Sipko y A. Bacchelli, “Modern Code Review: A Case Study at Google,” en *Proc. 40th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Gothenburg, Sweden: ACM, 2018, pp. 181–190. doi: 10.1145/3183519.3183525.
- [3] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017. ISBN: 978-0-13-449416-6.
- [4] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

- [5] T. Cerny, A. S. Abdelfattah, A. A. Maruf, A. Janes y D. Taibi, “Catalog and Detection Techniques of Microservice Anti-Patterns and Bad Smells: A Tertiary Study,” *Journal of Systems and Software*, vol. 206, p. 111829, 2023. doi: 10.1016/j.jss.2023.111829.
- [6] R. Tighilt, M. Abdellatif, I. Trabelsi, L. Madern, N. Moha e Y.-G. Gueheneuc, “On the Maintenance Support for Microservice-Based Systems through the Specification and the Detection of Microservice Antipatterns,” *Journal of Systems and Software*, vol. 204, p. 111755, 2023. doi: 10.1016/j.jss.2023.111755.
- [7] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018. ISBN: 978-1-6172-9454-9.
- [8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, USA: O’Reilly Media, 2021. ISBN: 978-1-4920-3402-5.
- [9] D. Taibi y V. Lenarduzzi, “On the Definition of Microservice Bad Smells,” *IEEE Software*, vol. 35, n.º 3, pp. 56–62, 2018. doi: 10.1109/MS.2018.2141031.
- [10] T. I. Mohottige, A. Polyvyanyy, C. Fidge, R. Buyya y A. Barros, “Reengineering Software Systems into Microservices: State-of-the-Art and Future Directions,” *Information and Software Technology*, vol. 183, p. 107732, 2025. doi: 10.1016/j.infsof.2025.107732.
- [11] International Organization for Standardization, *ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*. Geneva, Switzerland: ISO, 2023.